

Milestone 1. Model selection and Architecture

Activity 1.1. Research and Select the Appropriate Generative AI Model

Activity 1.2. Define the architecture of the application

Activity 1.3. Set up the development Environment

Milestone 1. Model selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI models from Google's Gemini family for the FitBuddy application. This includes evaluating the performance and capabilities of multiple Gemini models to ensure they align with the application's core functionalities—such as generating personalized workout plans, updating them based on feedback, and providing tailored nutrition or recovery advice.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Understand the Project Requirements

FitBuddy's AI model needed to:

Generate structured 7-day workout plans

Provide concise and actionable nutrition tips

Update workout plans based on user feedback

Perform well in real-time, web-based interaction

Model Evaluation

We initially experimented with several Hugging Face models:

T5 & BART – For text generation (workout/nutrition)

DistilBERT – For potential intent classification

Bloom – For general purpose generation

While these models showed promise in basic text generation, they had notable limitations.

These models required:

Hosting with Hugging Face or local GPU setup

Fine-tuning or complex prompting to achieve structured plans

Final Model Selection

After comparative testing, we selected Google Gemini models for production:

Gemini 1.5 Pro

- Used for structured 7-day workout generation and plan updating
- Delivers rich, context-aware, formatted responses reliably

Gemini Flash

- Used for generating fast, practical nutrition tips
- Lightweight and efficient for quick API calls

Reasons for Choosing Gemini:

API-based, easy to integrate with FastAPI

Naturally understands structured formatting (days, sections)

High accuracy with minimal prompt tuning

Scalable and fast for feedback-based revisions

Activity 1.2. Define the architecture of the application

Architecture Overview: FitBuddy follows a modular FastAPI-based architecture-

Frontend: HTML + Jinja2 templates for user input and display

Backend: FastAPI for route handling and logic

AI Layer: Google Gemini APIs (Pro & Flash)

Database: SQLite with SQLAlchemy for persistence

Frontend Responsibilities: Users interact via-

index.html – Input form

result.html – Plan, tip, feedback

all_users.html – Admin panel

Backend Responsibilities:

Receives form data

Calls Gemini models

Stores user info and plans

Renders dynamic templates

AI Integration Points:

Workout generation & feedback updates → Gemini Pro

Nutrition tips → Gemini Flash

Activity 1.3 Set up the development Environment:

Install Python & Pip: Ensure Python and pip are installed.

Create Virtual Environment:

```
python -m venv fitbuddy-env
```

```
fitbuddy-env\Scripts\activate (Windows)
```

Install Required Libraries:

```
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai
```

Run Server:

```
uvicorn app.main:app --reload
```

Visit <http://127.0.0.1:8000> for the app and /docs for API testing.

Set Up Project Structure:

Create folders: /app (Python files), /templates (HTML), /static/images (assets)

```
fitbuddy/
|
|— requirements.txt    # Project dependencies
|
|— app/
|   |— main.py        # FastAPI entry point
|   |— routes.py      # Core route handlers
|   |— database.py    # SQLAlchemy models and DB logic
|   |— schemas.py     # Pydantic models for validation
|   |— gemini_generator.py # Gemini Pro - workout plan generator
|   |— gemini_flash_generator.py # Gemini Flash - nutrition tips
|   |— updated_plan.py # Feedback-based plan updater
|   └─ nutrition.py   # Handles nutrition-specific logic (optional)
|
|— templates/
|   |— index.html     # User input form
|   |— result.html    # Workout plan, tip, feedback
|   └─ all_users.html # Admin dashboard
|
|— static/
|   └─ images/
|       └─ gym-bg.jpg # Gym-themed background
|
└─ fitbuddy.db        # Local SQLite database (auto-generated)
```